

Galaxy-X-os — One-Page Technical Report

SCALE x ODYSSEY | Deep-learning classification of raw astronomical images into 5 celestial categories. **GitHub:** <https://github.com/Srujan0798/Galaxy-X-os>

Dataset Sources

- **SDSS DR17** (Sloan Digital Sky Survey, Data Release 17) — real astronomical imaging used for the final trained model.
- Merged Kaggle galaxy-morphology datasets (Galaxy Zoo / DeepSky / Planetary imagery) for class balancing.
- **5 classes:** Spiral Galaxy, Elliptical Galaxy, Nebula, Star Cluster, Planetary Object.
- **Split:** 80 / 10 / 10 (train / val / test). ~2000 training images.

Model Architecture

- **Backbone:** EfficientNet-B3 (ImageNet pretrained, transfer learning) — ~12M parameters.
- **Head:** Replaced classifier → 5-class output.
- **Training strategy:** Progressive unfreezing (backbone frozen first 3 epochs), mixed-precision (`torch.amp`), Adam (lr `3e-4`), batch size 32.
- **Augmentations:** Albumentations astro-specific pipeline (cosmic-ray simulation, vignetting, Poisson noise, flips/rotations).
- **Explainability:** Grad-CAM heatmaps over predictions.

Final Test Metrics

Metric	Value
Test Accuracy	69.6%
Macro F1	0.684
Best Val Accuracy	75.0%

Per-class F1:

Class	F1
Elliptical Galaxy	1.00
Nebula	0.99
Planetary Object	0.60
Spiral Galaxy	0.43

Class	F1
Star Cluster	0.40

Strongest on Elliptical/Nebula; Spiral vs Star Cluster confusion drives the lower macro score (visually similar diffuse structures, limited training samples).

Confusion Matrix

See [results/confusion_matrix.png](#) . Additional plots: [results/per_class_metrics.png](#) , [results/training_curves.png](#) .

Inference Time

- **< 15 ms per image on GPU** (single forward pass, EfficientNet-B3, mixed precision).
- CPU inference runs in real time for the Streamlit demo.

Setup Instructions

```
```bash conda create -n scale_odyssey python=3.10 -y conda activate scale_odyssey pip install -r requirements.txt
```

```
python src/download_datasets.py # prepare data python src/train.py # train python src/evaluate.py # evaluate (standard + TTA) python src/gradcam.py # Grad-CAM report streamlit run app/app.py # web demo ```
```

## Training Script

---

Main entry point: [src/train.py](#) . Config: [config/config.yaml](#) . Model checkpoints saved to [checkpoints/best\\_model.pth](#) .